

BitFlip

CloudSherpa

Software Architecture Specifications

Contents

1	Introduction	2
2	Architectural Requirements	2
2.1	Architectural Patterns	2
2.2	Design Patterns	2
2.3	Constraints	2
2.4	Architectural Diagram	3
2.5	Mapping Quality Requirements to Architectural Decisions	3
3	Technology Requirements	8
3.1	Next.js	8
3.2	Spring Boot	9
3.3	Docker	10
3.4	PostgreSQL & TimescaleDB	11
3.5	TBD/WIP: AI technologies	11
4	API Contracts	11
4.1	Ingestion Service	11
5	Deployment	21
5.1	Deployment Requirements	21
5.2	Deployment Diagram	24
5.3	CI/CD Pipeline Diagram	25

1 | Introduction

The CloudSherpa architecture emphasizes separation of concerns, reliability and scalability.

2 | Architectural Requirements

2.1 Architectural Patterns

2.1.1 Service Oriented Architecture

CloudSherpa follows the Service Oriented Architecture (SOA) pattern, identifying three distinct services according to domain boundaries, namely:

- **Ingestion Service**
The ingestion service is responsible for connecting to the respective cloud provider interfaces for obtaining data from deployments.
- **Service Service**
The Service Service is responsible for the business logic of CloudSherpa, executing tasks like data aggregation, authentication and communicating intelligence insights to the Frontend Service.
- **Frontend Service**
The Frontend Service is responsible for tasks related to the CloudSherpa User Interface.

2.2 Design Patterns

2.3 Constraints

2.3.1 Supported Cloud Providers

The system shall support AWS, Azure and GCP as cloud providers. Cloud providers not listed are outside of the scope of CloudSherpa.

2.3.2 Secure handling of cloud credentials

The system must follow the principle of least privilege when monitoring deployments accross cloud providers.

2.4 Architectural Diagram

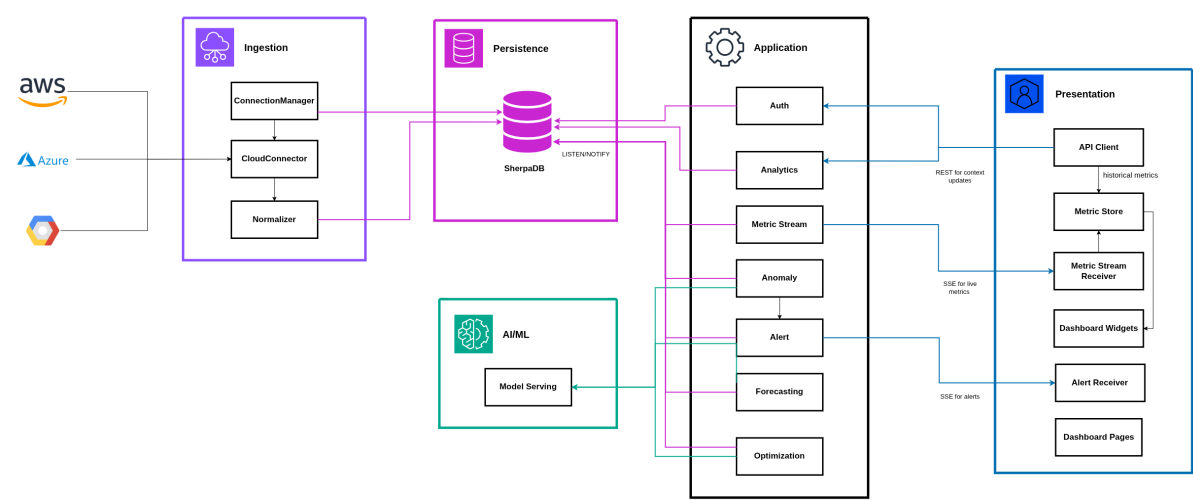


Figure 1: Architectural Diagram

2.5 Mapping Quality Requirements to Architectural Decisions

Table 1: Mapping Quality Requirements to Architectural Decisions

NFR1: Performance	

NFR1.1: The system should load the main dashboard within 2 seconds and update visualization widgets within 1 second of a user modifying a time window, while seamlessly handling the throughput required to ingest metrics from 1,000 live deployments.	The ingestion service, which is responsible for processing incoming cloud metrics, is deployed independently from the service layer that handles dashboard requests for historical data and propagates live updates to the user interface. This separation prevents ingestion workloads from directly competing with dashboard requests for the same application resources, provided that the deployment allocates sufficient resources to each service. To support the requirement that the main dashboard load within two seconds and remain responsive, the system preprocesses and aggregates data before sending it to the dashboard, thereby reducing the number of data points that must be transferred and rendered. Once the dashboard receives the data, it is stored centrally and reused across multiple widgets rather than being duplicated, which reduces memory usage and avoids unnecessary processing.
NFR1.2: The ingestion pipeline shall process, normalize, and aggregate 100,000 raw billing and usage records from AWS, Azure, and GCP APIs into the unified internal schema within 60 seconds of data retrieval.	The architecture allows the ingestion service, which is responsible for processing and normalizing cloud billing and usage records, to be scaled independently according to ingestion volume and processing demand, hence improving performance by allocating sufficient resources.
NFR2: Scalability	

NFR2.1: The system should successfully support 500 concurrent dashboard users and manage 1,000 active cloud connections without experiencing more than a 10% degradation in dashboard response times.	Data ingestion, processing, and normalization are performed by the independently deployable ingestion service, while dashboard requests are handled by the service layer. The service layer retrieves and aggregates historical data into a form optimized for dashboard rendering, reducing the amount of data transferred and processed for each user request. Because the ingestion and user-facing services are decoupled and operate in isolated execution environments, increases in the number of connected cloud accounts primarily affect the ingestion service rather than the dashboard request path. Each service can therefore be scaled independently according to its workload, helping the system support additional cloud connections and concurrent dashboard users without a significant degradation in response time.
NFR2.2: The time-series database architecture shall scale dynamically to accommodate a 200% year-over-year growth in stored historical cloud data without requiring manual partitioning or major architectural changes.	???
NFR3: Security	

NFR3.1: All sensitive user data and stored cloud provider credentials must be encrypted using TLS 1.3 in transit, and the system must enforce Multi-Factor Authentication (MFA) on 100% of accounts.	All external requests pass through a single controlled entry point before reaching the system's internal services. TLS 1.3 is enforced at this boundary, ensuring that communication between the dashboard and backend services is encrypted in transit. Authentication is centralised through a shared identity and access management component, which requires multi-factor authentication for all user accounts and prevents unauthenticated requests from bypassing the authorisation process.
NFR3.2: The system shall utilize secure, least-privilege identity access management (IAM) strategies, ensuring that CloudSherpa holds read-only access to client AWS, Azure, and GCP billing APIs and ensuring that none of CloudSherpa's requests require write permissions.	Third-party integrations are confined to a single service, allowing them to be implemented through consistent interfaces, governed by provider-specific security configurations, and monitored and audited from a central point.
NFR4: Reliability	
NFR4.1: The system should achieve 99.9% uptime for both the dashboard and the metrics ingestion pipeline.	<i>Note from Gerard: Need to incorporate high availability design principles into our architecture, database clustering, load balancing and replicated instances, circuit breaker pattern. Our current architecture is not reliable and needs to be revised with reliability in mind</i>
NFR5: Maintainability	
NFR5.1: The codebase must maintain a minimum of 80% automated test coverage across all core components, and the continuous integration pipeline should enable new feature deployments or bug fixes to reach production within 2 hours.	The modularity of our architecture improves the ease and pace at which tests can be developed

NFR5.2: The data normalization module must be sufficiently decoupled so that integrating a new cloud provider API, or updating an existing provider's changed billing schema, requires no more than 40 hours of developer effort.	The ingestion service, which is responsible for data ingestion and normalization, is decoupled from the rest of the system. Changes to this service do not require corresponding changes to other services, because communication between the ingestion and service layers occurs through a shared database schema that provides a standardized data contract.
NFR6: Usability	
NFR6.1: A new non-technical user should be able to complete core tasks, such as viewing forecasts or checking anomaly alerts, within 5 minutes of first using the system, achieving at least 85% user satisfaction during usability testing.	As the presentation service is decoupled from how the data is obtained from the Cloud providers, developers can engineer an intuitive user experience removed from the provider specific complexities.
NFR6.2: The CloudSherpa dashboard shall render correctly on modern web browsers to ensure plain-language summaries remain highly accessible.	
NFR7: Availability	
NFR7.1: The system should be available 24/7, excluding scheduled maintenance periods not exceeding 2 hours per month.	<i>Note from Gerard: Need to incorporate high availability design principles into our architecture, database clustering, load balancing and replicated instances, circuit breaker pattern. Our current architecture is not reliable and needs to be revised with reliability in mind</i>

NFR7.2: The web dashboard and historical cost databases shall remain available and accessible to users even if one or more underlying cloud provider APIs (AWS, Azure, or GCP) experience a temporary outage.	Ingested cloud data is persisted in a database managed as part of the CloudSherpa deployment. The dashboard therefore retrieves historical cost and usage data from the local persistence layer rather than directly from third-party cloud provider APIs. As a result, temporary degradation or unavailability of AWS, Azure, or GCP APIs does not prevent users from accessing previously ingested historical data.

3 | Technology Requirements

3.1 Next.js

We chose to use the Next.js full-stack framework for the CloudSherpa user-facing **Dashboard**.

Please note that references such as **FR X** or **NFR X** refer to requirements documented in the CloudSherpa Software Requirement Specification documentation.

3.1.1 Supporting functional requirements

- **FR 5. Web-Based Dashboard**

Next.js is a framework intended for web development and thus facilitates the development of our Web-Based Dashboard as elicited by FR 5 in our SRS document.

- **FR 6. User and Account Management**

Next.js is built on top of the React framework and thus enables developers to perform advanced and reactive state management. This feature enables the development of user facing user and account management functionality.

3.1.2 Supporting non-functional requirements

- **NFR 3. Security**
Next.js Server Components hide sensitive information and code from the client.
- **Performance**
Next.js Server Components can be leveraged for computationally intensive Dashboard tasks, limiting client-side computation and improving the dashboard response times.
- **NFR 7. Scalability**
Since our dashboard aggregates large amounts of data, Next.js server components can be leveraged to improve load tolerance by employing means such as
 - a Server-Side cache for historical data
- **NFR 6 Usability**
Next.js integrates with component libraries like ShadCN optimized for Dashboard usability and accessibility.

3.1.3 Ease of development

- **App Router**
The Next.js App Router simplifies development by utilizing directory structure to define app navigation.

3.2 Spring Boot

Most of the CloudSherpa business logic is facilitated by the Spring Boot framework using Java 21.

3.2.1 Supporting functional requirements

- **FR 1. Cloud Data Integration**
Utilizing spring scheduler, ingest cloud data at regular intervals
- **FR 2. Data storage and processing**
Spring Boot integrates with PostgreSQL Notify/Listen to perform business logic, i.e. streaming metrics, initialized by a Database write.
- **FR 6. User and Account Management**
Utilizing Spring Security Filter Chain for managing authorization, users and user-related context can be obtained for each request as part of the Authentication Context facilitated by Spring Security and the JWT protocol.

3.2.2 Supporting non-functional requirements

- **NFR 1. Performance**
Multi-threaded architecture allows for competitive performance.
- **NFR 3. Security**
Spring Security provides a pipe-and-filter architecture to ensure that unauthenticated or unauthorized requests never reach the controllers but gets filtered out at an earlier stage in the Filter Chain.
- **NFR 5. Maintainability**
OOP and Java packages enables modular development making code easier to test, maintain and extend. Supporting testing frameworks such as *JUnit*, *Mockito* and *AssertJ* enable unit and integration testing.

3.2.3 Ease of development

3.3 Docker

3.3.1 Supporting non-functional requirements

- **Security**
Our application runs as a set of isolated containers. Isolated containers have the benefit of increased security: when one part of the system is compromised it does not automatically mean that the entire system is compromised. Taking this into account, we explicitly handle all sensitive operations within a part of the system that runs in an isolated container and is less likely to be vulnerable to malicious exploits.
- **Performance**
Docker shares the operating system kernel which results in better performance than using traditional VMs.
- **Scalability**
Containers can easily be scaled up or down. Containers can be replicated to facilitate distributed runtime environments.

3.3.2 Ease of development

- **Portability**
Docker was chosen to facilitate portability of the application such that it is not dependent on the environment in which it is executed.
- **Reproducibility**
Ideally, the environment in which the containers are run should not affect the

container behaviour. Containers are isolated from the host, and thus should behave similarly accross environments yielding the benefit of reproducible runs.

3.4 PostgreSQL & TimescaleDB

We use PostgreSQL as our RDBMS along with the PostgreSQL extension for storing time-series data, TimescaleDB. A big part of our reasoning behind choosing PostgreSQL as a RDBMS is the fact that TimescaleDB is built on top of PostgreSQL and integrates with non-hypertable relational tables in one DB.

3.4.1 Supporting functional requirements

- **FR 2. Data Storage and Processing**
PostgreSQL schemas define how data is stored at rest. We use the PostgreSQL extension TimescaleDB for storing time-series data. Built in PostgreSQL NOTIFY/LISTEN functionality used

3.4.2 Supporting non-functional requirements

- **Performance**
TimescaleDB optimizes the storage and query efficiency of time-series data

3.5 TBD/WIP: AI technologies

4 | API Contracts

4.1 Ingestion Service

`/api/events/ingest`

Ingest real cloud usage and billing data

Request:

```
1 {  
2   "scopes": [  
3     {  
4       "provider": "string",
```

```
5     "accountId": "string",
6     "subscriptionId": "string",
7     "projectId": "string",
8     "billingAccountId": "string",
9     "serviceScopes": [
10    {
11        "name": "string",
12        "instances": [
13            {
14                "identifierName": "string",
15                "values": [
16                    "string"
17                ]
18            }
19        ],
20        "metrics": [
21            {
22                "name": "string",
23                "unit": "string"
24            }
25        ]
26    }
27 ]
28 }
29 ],
30 "credentials": {
31     "accessKey": "string",
32     "secretKey": "string",
33     "awsRegion": "string",
34     "tenantId": "string",
35     "clientId": "string",
36     "clientSecret": "string",
37     "projectId": "string",
38     "serviceAccountJson": "string"
39 },
40 "from": "2026-06-28T20:17:14.812Z",
41 "to": "2026-06-28T20:17:14.812Z",
42 "period": 0,
43 "includeBilling": true,
44 "includeUsage": true,
45 "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
46 }
```

Response:

```
1 {
2   "usage": [
3     {
4       "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
5       "provider": "string",
6       "accountId": "string",
7       "subscriptionId": "string",
8       "projectId": "string",
9       "resourceId": "string",
10      "resourceType": "string",
11      "serviceName": "string",
12      "region": "string",
13      "metricName": "string",
14      "value": 0.1,
15      "unit": "string",
16      "timestamp": "2026-06-28T20:17:14.814Z",
17      "periodStart": "2026-06-28T20:17:14.814Z",
18      "periodEnd": "2026-06-28T20:17:14.814Z",
19      "dimensions": {
20        "additionalProp1": "string",
21        "additionalProp2": "string",
22        "additionalProp3": "string"
23      },
24      "tags": {
25        "additionalProp1": "string",
26        "additionalProp2": "string",
27        "additionalProp3": "string"
28      },
29      "ingestionTimestamp": "2026-06-28T20:17:14.814Z",
30      "ingestionId": "string",
31      "source": "string"
32    }
33  ],
34  "billing": [
35    {
36      "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
37      "provider": "string",
38      "accountId": "string",
39      "subscriptionId": "string",
40      "projectId": "string",
41      "billingAccountId": "string",
42      "serviceName": "string",
```

```

43     "resourceId": "string",
44     "resourceType": "string",
45     "region": "string",
46     "cost": 0.1,
47     "usageQuantity": 0.1,
48     "unit": "string",
49     "currency": "string",
50     "pricingModel": "string",
51     "usageStartTime": "2026-06-28T20:17:14.814Z",
52     "usageEndTime": "2026-06-28T20:17:14.814Z",
53     "billingPeriodStart": "2026-06-28T20:17:14.814Z",
54     "billingPeriodEnd": "2026-06-28T20:17:14.814Z",
55     "tags": {
56         "additionalProp1": "string",
57         "additionalProp2": "string",
58         "additionalProp3": "string"
59     },
60     "ingestionTimestamp": "2026-06-28T20:17:14.814Z",
61     "ingestionId": "string",
62     "source": "string"
63 }
64 ]
65 }

```

/api/events/ingest/mock

Generates and ingests deterministic mock cloud usage data for development and integration testing.

Request:

```

1 {
2   "scopes": [
3     {
4       "provider": "string",
5       "accountId": "string",
6       "subscriptionId": "string",
7       "projectId": "string",
8       "billingAccountId": "string",
9       "serviceScopes": [
10        {
11          "name": "string",
12          "instances": [

```

```

13         {
14             "identifierName": "string",
15             "values": [
16                 "string"
17             ]
18         }
19     ],
20     "metrics": [
21         {
22             "name": "string",
23             "unit": "string"
24         }
25     ]
26 }
27 ]
28 }
29 ],
30 "credentials": {
31     "accessKey": "string",
32     "secretKey": "string",
33     "awsRegion": "string",
34     "tenantId": "string",
35     "clientId": "string",
36     "clientSecret": "string",
37     "projectId": "string",
38     "serviceAccountJson": "string"
39 },
40 "from": "2026-06-28T20:20:54.560Z",
41 "to": "2026-06-28T20:20:54.560Z",
42 "period": 0,
43 "includeBilling": true,
44 "includeUsage": true,
45 "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
46 }

```

Response:

```

1 {
2     "usage": [
3         {
4             "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
5             "provider": "string",
6             "accountId": "string",

```

```
7     "subscriptionId": "string",
8     "projectId": "string",
9     "resourceId": "string",
10    "resourceType": "string",
11    "serviceName": "string",
12    "region": "string",
13    "metricName": "string",
14    "value": 0.1,
15    "unit": "string",
16    "timestamp": "2026-06-28T20:20:54.565Z",
17    "periodStart": "2026-06-28T20:20:54.565Z",
18    "periodEnd": "2026-06-28T20:20:54.565Z",
19    "dimensions": {
20      "additionalProp1": "string",
21      "additionalProp2": "string",
22      "additionalProp3": "string"
23    },
24    "tags": {
25      "additionalProp1": "string",
26      "additionalProp2": "string",
27      "additionalProp3": "string"
28    },
29    "ingestionTimestamp": "2026-06-28T20:20:54.565Z",
30    "ingestionId": "string",
31    "source": "string"
32  }
33 ],
34 "billing": [
35   {
36     "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
37     "provider": "string",
38     "accountId": "string",
39     "subscriptionId": "string",
40     "projectId": "string",
41     "billingAccountId": "string",
42     "serviceName": "string",
43     "resourceId": "string",
44     "resourceType": "string",
45     "region": "string",
46     "cost": 0.1,
47     "usageQuantity": 0.1,
48     "unit": "string",
49     "currency": "string",
50     "pricingModel": "string",
```

```
51     "usageStartTime": "2026-06-28T20:20:54.565Z",
52     "usageEndTime": "2026-06-28T20:20:54.565Z",
53     "billingPeriodStart": "2026-06-28T20:20:54.565Z",
54     "billingPeriodEnd": "2026-06-28T20:20:54.565Z",
55     "tags": {
56         "additionalProp1": "string",
57         "additionalProp2": "string",
58         "additionalProp3": "string"
59     },
60     "ingestionTimestamp": "2026-06-28T20:20:54.565Z",
61     "ingestionId": "string",
62     "source": "string"
63 }
64 ]
65 }
```

/api/events/ingest/mockNoise

Generates and ingests mock cloud usage data using Reinstein Uhlenbeck noise for analytics, anomaly detection, and testing.

Request:

```
1 {
2   "scopes": [
3     {
4       "provider": "string",
5       "accountId": "string",
6       "subscriptionId": "string",
7       "projectId": "string",
8       "billingAccountId": "string",
9       "serviceScopes": [
10        {
11          "name": "string",
12          "instances": [
13            {
14              "identifierName": "string",
15              "values": [
16                "string"
17              ]
18            }
19          ],
20          "metrics": [
```

```
21         {
22             "name": "string",
23             "unit": "string"
24         }
25     ]
26 }
27 ]
28 }
29 ],
30 "credentials": {
31     "accessKey": "string",
32     "secretKey": "string",
33     "awsRegion": "string",
34     "tenantId": "string",
35     "clientId": "string",
36     "clientSecret": "string",
37     "projectId": "string",
38     "serviceAccountJson": "string"
39 },
40 "from": "2026-06-28T20:23:56.974Z",
41 "to": "2026-06-28T20:23:56.974Z",
42 "period": 0,
43 "includeBilling": true,
44 "includeUsage": true,
45 "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
46 }
```

Response:

```
1 {
2     "usage": [
3         {
4             "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
5             "provider": "string",
6             "accountId": "string",
7             "subscriptionId": "string",
8             "projectId": "string",
9             "resourceId": "string",
10            "resourceType": "string",
11            "serviceName": "string",
12            "region": "string",
13            "metricName": "string",
14            "value": 0.1,
```

```
15     "unit": "string",
16     "timestamp": "2026-06-28T20:23:56.980Z",
17     "periodStart": "2026-06-28T20:23:56.980Z",
18     "periodEnd": "2026-06-28T20:23:56.980Z",
19     "dimensions": {
20         "additionalProp1": "string",
21         "additionalProp2": "string",
22         "additionalProp3": "string"
23     },
24     "tags": {
25         "additionalProp1": "string",
26         "additionalProp2": "string",
27         "additionalProp3": "string"
28     },
29     "ingestionTimestamp": "2026-06-28T20:23:56.980Z",
30     "ingestionId": "string",
31     "source": "string"
32 }
33 ],
34 "billing": [
35     {
36         "recordId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
37         "provider": "string",
38         "accountId": "string",
39         "subscriptionId": "string",
40         "projectId": "string",
41         "billingAccountId": "string",
42         "serviceName": "string",
43         "resourceId": "string",
44         "resourceType": "string",
45         "region": "string",
46         "cost": 0.1,
47         "usageQuantity": 0.1,
48         "unit": "string",
49         "currency": "string",
50         "pricingModel": "string",
51         "usageStartTime": "2026-06-28T20:23:56.980Z",
52         "usageEndTime": "2026-06-28T20:23:56.980Z",
53         "billingPeriodStart": "2026-06-28T20:23:56.980Z",
54         "billingPeriodEnd": "2026-06-28T20:23:56.980Z",
55         "tags": {
56             "additionalProp1": "string",
57             "additionalProp2": "string",
58             "additionalProp3": "string"
```

```
59     },
60     "ingestionTimestamp": "2026-06-28T20:23:56.980Z",
61     "ingestionId": "string",
62     "source": "string"
63   }
64 ]
65 }
```

`/api/cloud-resources/services`

Retrieves a list of all services that can be monitored by CloudSherpa for a given provider

Request:

```
1 {
2   "string"
3 }
```

Response:

```
1 {
2   "string"
3 }
```

`/api/cloud-resources/resources/provider`

Retrieves all discoverable resources for a cloud provider using the supplied credentials

Request:

```
1 {
2   "accessKey": "string",
3   "secretKey": "string",
4   "awsRegion": "string",
5   "tenantId": "string",
6   "clientId": "string",
7   "clientSecret": "string",
8   "projectId": "string",
9   "serviceAccountJson": "string"
10 }
```

Response:

```
1 {
2   "resourceId": "string",
3   "name": "string",
4   "resourceType": "string",
5   "serviceCategory": "string",
6   "tags": {
7     "additionalProp1": "string",
8     "additionalProp2": "string",
9     "additionalProp3": "string"
10  }
11 }
```

/api/cloud-resources/aws/permissions

Generates a least-privilege AWS IAM policy for the selected services

Request:

```
1 [
2   "string"
3 ]
```

Response:

```
1 {
2   string
3 }
```

5 | Deployment

5.1 Deployment Requirements

- **Live, Accessible System:**

The system is accessible at <https://cloudsherpa.gjjcs.org>

- **Environment Parity:**

- **Development Environment**

- BitFlip members develop CloudSherpa on their local machines. To ensure reproducibility and consistency across the team, the development environment is containerized using Docker and Docker Compose. This setup closely mirrors the production deployment environment, particularly in how container images are built and services are orchestrated. Team members who do not have sufficient local computing resources to run the complete containerized environment can instead make use of the development servers provided by the frameworks that we use (refer to section 3).

- **Staging Environment**

- The CloudSherpa staging environment is hosted on an AWS EC2 instance, with deployment automated through GitHub Actions. It serves as a Quality Assurance (QA) environment in which the team can test the platform under conditions that closely resemble the production environment. Nginx-enforced HTTP Basic Authentication restricts access to authorized team members and stakeholders, preventing unintended public access to the staging deployment. The automated deployment workflow is triggered by a push event to the `staging` branch.

- **Production Environment**

- The CloudSherpa production environment is hosted on an AWS EC2 instance, with deployment automated through GitHub Actions. The production deployment workflow is triggered by a push event to the `main` branch.

- **Containerisation:**

Docker was selected as the containerisation platform for CloudSherpa, enabling the system to be developed and deployed as a set of isolated, portable services. Docker Compose is used to define and manage the platform's multi-container architecture. For deployment, AWS Elastic Container Registry (ECR) serves as a private, managed container registry that integrates seamlessly with the staging and production environments hosted on AWS.

- **Secrets Management:**

Local `.env` files are used to manage secrets and environment-specific configuration. To ensure consistency across environments and enable configuration validation, the project uses environment variable schemas. *Varlock* was selected to define and manage these schemas, together with a custom validation script that verifies that all required environment variables are present. The Varlock schema files can be safely committed to version control because they define only the expected configuration structure and do not contain any secret values.

- **Rollback Strategy:**

CloudSherpa container images are built and deployed automatically using GitHub Actions. Each image is tagged with the commit hash associated with the deployment, allowing deployments to be pinned to specific versions of the codebase. A deployment script accepts a commit hash as a command-line argument, pulls the corresponding images from the container registry, and deploys them. If a rollback is required, the commit hash of a previously stable deployment is supplied to the script, restoring the system to that version.

5.2 Deployment Diagram

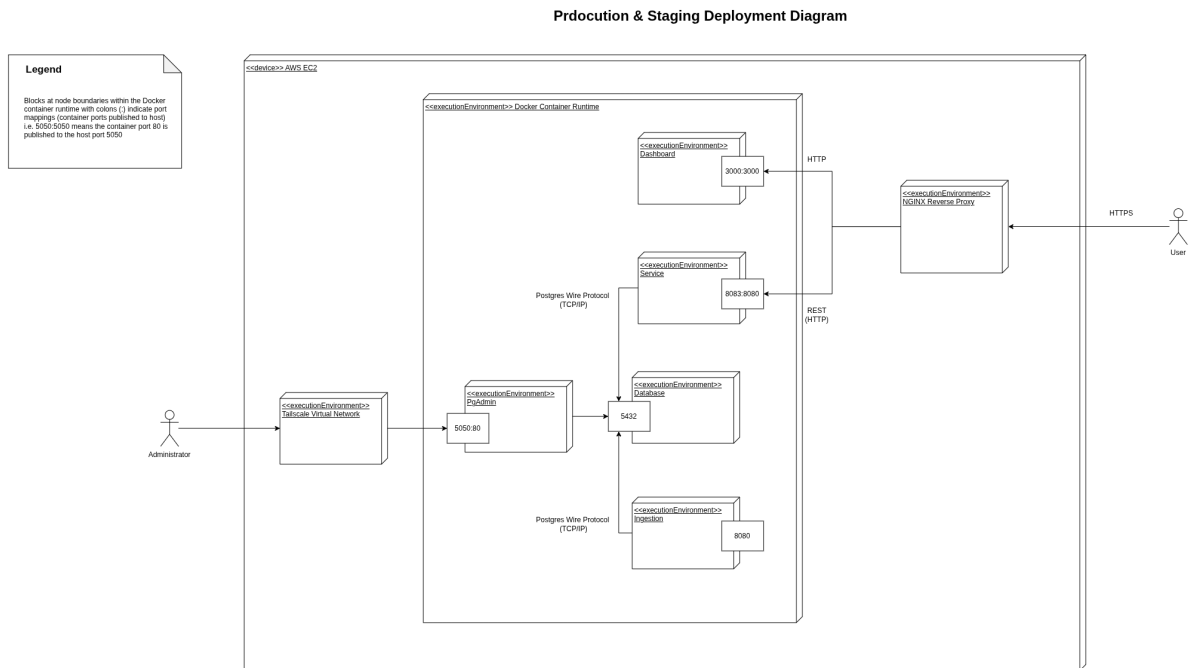


Figure 2: CloudSherpa Deployment Diagram (Production)

5.3 CI/CD Pipeline Diagram

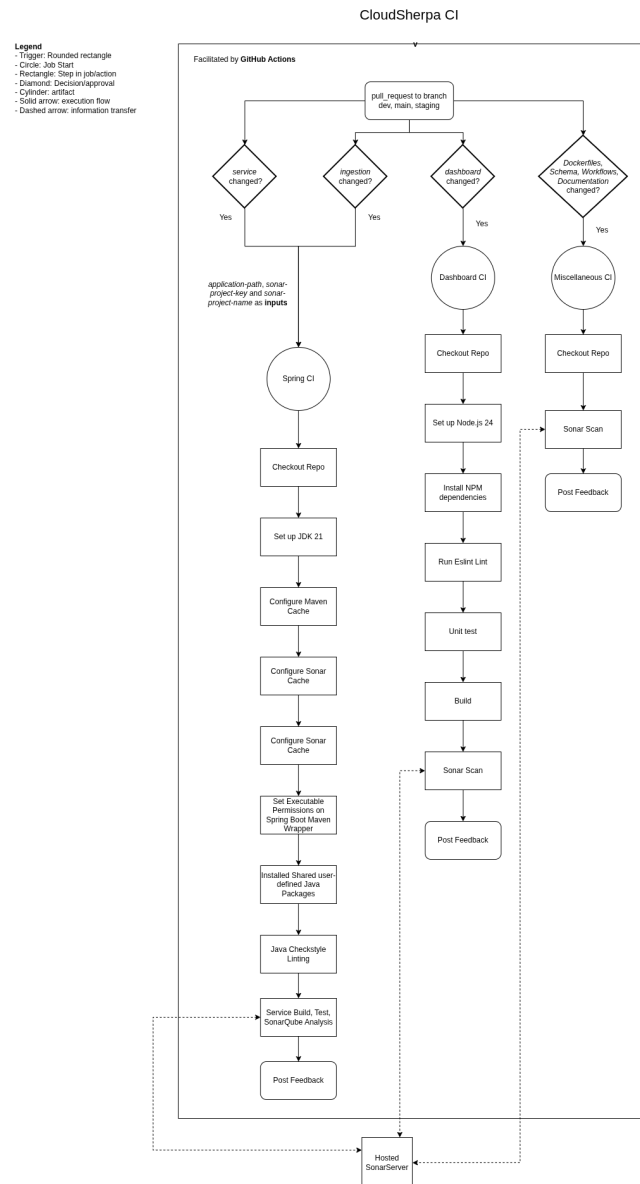


Figure 3: CI Pipeline Flow

CloudSherpa CD

Legend
 - Trigger: Rounded rectangle
 - Circle: Job Start
 - Rectangle: Step in job/action
 - Diamond: Decision/Approval
 - Cylinder: Artifact
 - Solid arrow: execution flow
 - Dashed arrow: information transfer

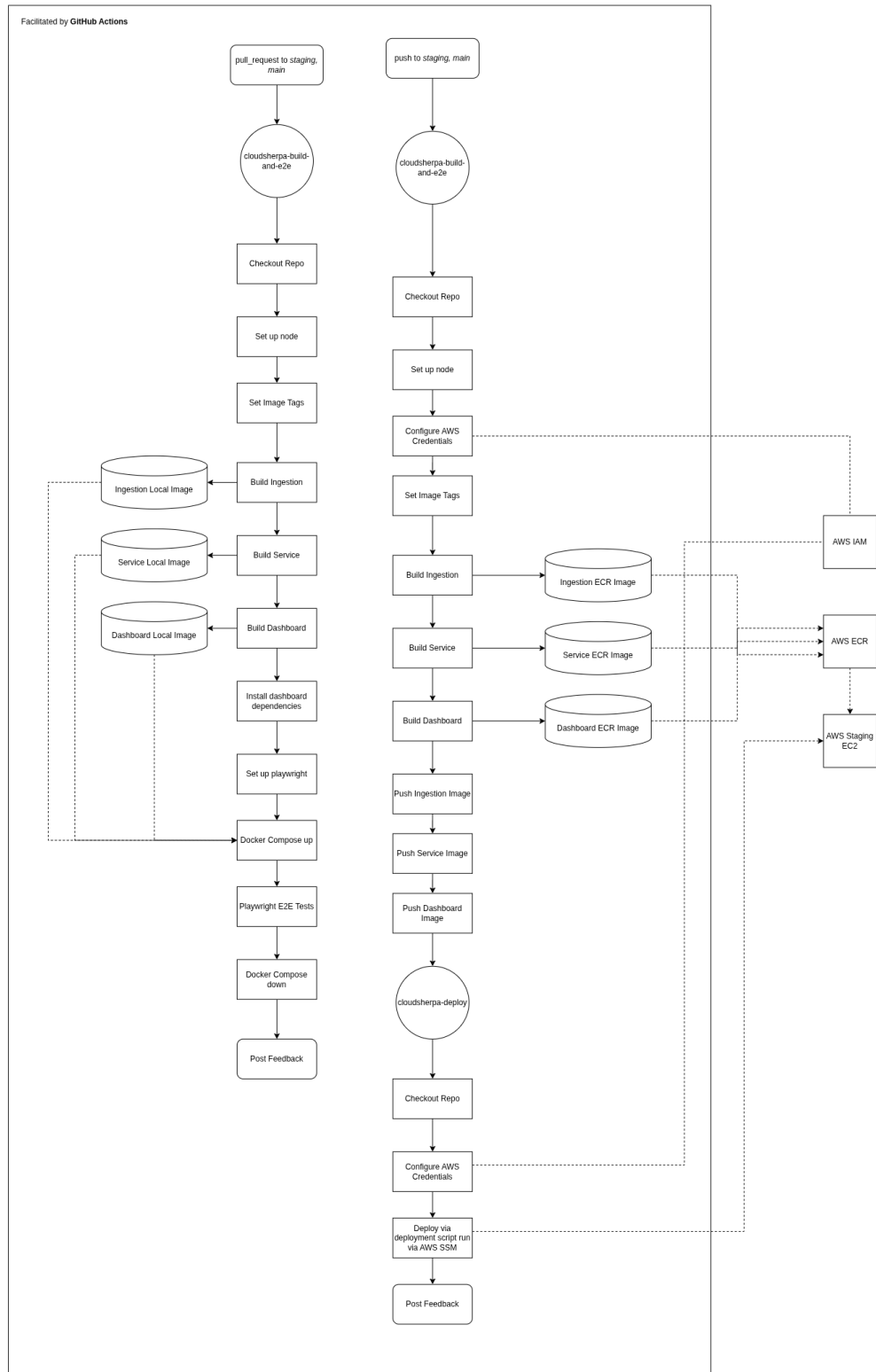


Figure 4: CD Pipeline Flow